

개요

본 문서는 CSCI103 Lecture 4 강의 자료를 바탕으로 데이터 변환(Data Transformations)에 대한 핵심 개념을 다룹니다. 빅데이터 설계 패턴, 데이터 규정 준수, Spark 내부 동작 원리, Delta Lake 기능, 사용자 정의 함수(UDF), CRUD 작업, 스키마 진화, 변경 데이터 캡처(CDC) 및 서서히 변하는 차원(SCD), 그리고 Autoloader를 활용한 데이터 로딩 패턴까지 폭넓게 설명합니다. 이 문서는 비전공자나 처음 배우는 사람도 완벽히 이해할 수 있도록 각 개념의 필요성부터 실제 적용 방법까지 상세한 설명과 예시를 제공하여 시험 대비 및 실무 활용에 도움을 주는 것을 목표로 합니다.

1 용어 정리

다음 표는 본 강의에서 다루는 주요 용어들을 쉽게 설명하고 원어를 병기하여 이해를 돕습니다.

용어	쉬운 설명	원어	비고
스트리밍 (파일 기반)	파일이 도착하는 대로 처리하는 방식	File-based Streaming	
스트리밍 (이벤트 기반)	이벤트가 발생할 때마다 처리하는 방식	Event-based Streaming	
체크포인팅	장애 발생 시 마지막 저장 지점으로 복구하는 기능	Checkpointing	
워터마킹	너무 오래된 데이터를 폐기하는 기준 설정	Watermarking	데이터 지연 처리
람다 아키텍처	배치 처리와 실시간 스트리밍을 결합한 데이터 처리 방식	Lambda Architecture	
카파 아키텍처	모든 데이터를 단일 스트리밍 파이프라인으로 처리하는 방식	Kappa Architecture	
레이크하우스	데이터 웨어하우스와 데이터 레이크의 장점을 결합한 아키텍처	Lakehouse	
관리형 테이블	데이터와 메타데이터 모두 프레임워크가 관리하는 테이블	Managed Tables	
외부 테이블	데이터는 외부 저장소에, 메타데이터만 프레임워크가 관리하는 테이블	External Tables	
메타데이터	데이터에 대한 데이터 (설명, 위치, 소유자 등)	Metadata	데이터 거버넌스
구체화된 뷰	미리 계산되어 저장된 뷰 (빠르지만 오래될 수 있음)	Materialized View	
설계 패턴	재사용 가능한 문제 해결 템플릿	Design Patterns	
CQRS	쓰기(Command)와 읽기(Query) 책임을 분리하는 설계 패턴	Command Query Responsibility Segregation	
폴리글랏 영속성	데이터 유형에 따라 최적의 데이터베이스를 선택하는 전략	Polyglot Persistence	"적재적소의 도구"
데이터 레이크 패턴	원본 형태의 대량 데이터를 저장하는 패턴	Data Lake Pattern	
마이크로 배치 처리	스트리밍 데이터를 작은 배치 단위로 처리하는 방식	Microbatch Processing	준실시간 처리
CAP 이론	분산 시스템에서 일관성, 가용성, 분할 내성 중 두 가지만 선택 가능	CAP Theorem	
데이터 메시	분산된 데이터 도메인을 제품처럼 관리하고 공유하는 아키텍처	Data Mesh	
에이전트 아키텍처	LLM을 활용하여 도구와 에이전트를 조율하는 패턴	Agentic Architecture	최신 AI 활용
커넥터 패턴	다양한 데이터 소스에 일관된 방식으로 접근하는 패턴	Connector Pattern	브릿지 패턴
메달리온 아키텍처	데이터를 원본, 정제, 집계 단계로 나누어 처리하는 파이프라인	Medallion Architecture	멀티홉 패턴
시간 여행	데이터의 특정 시점 또는 버전으로 돌아가는 기능	Time Travel	Delta Lake 기능
제로 카피 클론	실제 데이터 복사 없이 메타데이터만 복사하여 데이터 복제	Zero Copy Clone	효율적인 복제
GDPR	유럽 연합의 일반 데이터 보호 규정	General Data Protection Regulation	개인 정보 보호
CCPA	캘리포니아 소비자 개인 정보 보호법	California Consumer Privacy Act	개인 정보 보호
삭제할 권리	소비자가 자신의 데이터를 시스템에서 삭제하도록 요구할 권리	Right of Erasure	
데이터 이동성 권리	소비자가 자신의 데이터를 다른 서비스로 이전하도록 요구할 권리	Right of Portability	
가명화/익명화	개인 식별 정보를 제거하거나 대체하여 프라이버시 보호	Pseudonymization/Anonymization	
Spark 작업/단계/태스크	Spark가 쿼리를 실행하는 논리적/물리적 단위 계층	Jobs, Stages, Tasks	병렬 처리 단위
테이블 파티셔닝	대규모 테이블을 관리하기 쉬운 작은 조각으로 나누는 방식	Table Partitioning	물리적 저장
Spark 파티셔닝	Spark가 데이터를 노드에 분산하여 처리하는 방식	Spark Partitioning	논리적 처리
파티션 가지치기	쿼리 시 불필요한 파티션 스캔을 건너뛰는 최적화	Partition Pruning	쿼리 성능 향상
데이터 셔플링	Spark 노드 간에 데이터를 재분배하는 작업 (비용 발생)	Data Shuffling	조인, 그룹화 시 발생
Z-오더링	관련 데이터를 물리적으로 가깝게 저장하여 쿼리 속도 향상	Z-Ordering	Delta Lake 최적화
브로드캐스트 해시 조인	작은 테이블을 모든 위커 노드에 복사하여 조인 속도 향상	Broadcast Hash Join	
셔플 해시 조인	한쪽 테이블이 다른 테이블보다 작고 메모리에 적합할 때 사용	Shuffle Hash Join	
정렬 병합 조인	모든 크기의 테이블에 적용 가능한 기본 조인 방식 (느릴 수 있음)	Sort Merge Join	
사용자 정의 함수 (UDF)	사용자가 직접 정의하는 함수 (한 행 입력, 한 행 출력)	User-Defined Function	
벡터화된 UDF	Pandas UDF를 사용하여 Spark와 Python 간 직렬화 오버헤드 감소	Vectorized UDF	성능 향상
사용자 정의 집계 함수 (UAF)	여러 행을 입력받아 단일 집계 값을 반환하는 함수	User-Defined Aggregate Function	SUM, COUNT 등
사용자 정의 변환 함수 (UDTF)	여러 행을 입력받아 여러 행을 출력하는 함수	User-Defined Transformative Function	EXPLODE, PIVOT 등
CRUD	데이터에 대한 기본 작업 (생성, 읽기, 업데이트, 삭제)	Create, Read, Update, Delete	
Upsert	데이터가 존재하면 업데이트하고, 없으면 삽입하는 작업	Upsert	MERGE 명령
스키마 추론	데이터로부터 스키마를 자동으로 감지하는 기능	Schema Inference	
스키마 진화	데이터 스키마 변경(새 컬럼 추가 등)을 허용하는 기능	Schema Evolution	mergeSchema=true
스키마 강제	정의된 스키마와 다른 데이터가 들어오면 오류를 발생시키는 기능	Schema Enforcement	
변경 데이터 캡처 (CDC)	원본 시스템에서 변경된 데이터를 감지하고 기록하는 기술	Change Data Capture	소스 측면
서서히 변하는 차원 (SCD)	차원 테이블의 변경 이력을 관리하는 방법	Slowly Changing Dimensions	타겟 측면
Autoloader	클라우드 저장소의 새로운 데이터를 점진적으로 처리하는 기능	Autoloader	스트리밍 파일 처리

2 핵심 개념 및 원리

2.1 지난 강의 복습

이번 강의는 지난 강의에서 다룬 핵심 개념들을 빠르게 복습하며 시작합니다. 이는 새로운 내용을 이해하기 위한 중요한 기반이 됩니다.

- 두 가지 스트리밍 유형:
 - 파일 기반 (File-based): 파일 시스템에 새로운 파일이 도착하는 것을 감지하여 처리합니다.
 - 이벤트 기반 (Event-based): 실시간으로 발생하는 개별 이벤트를 즉시 처리합니다.
- 체크포인팅 (Checkpointing):
 - 한 줄 요약: 장애 발생 시 데이터 처리의 연속성을 보장하기 위해 마지막으로 성공한 상태를 저장하는 기능입니다.
 - 직관적 비유: 게임을 하다가 중간에 저장하는 것과 같습니다. 게임이 멈추거나 꺼져도 마지막 저장 지점부터 다시 시작할 수 있습니다.
 - 기술적 설명: Spark 스트리밍과 같은 시스템에서 처리 상태(예: 읽은 오프셋)를 주기적으로 저장하여, 시스템 오류 발생 시 마지막 저장 지점부터 복구하여 데이터 손실 없이 처리를 재개할 수 있도록 합니다.
- 워터마킹 (Watermarking):
 - 한 줄 요약: 데이터의 지연 도착을 허용하면서도, 너무 오래된 데이터를 처리 파이프라인에서 제외하기 위한 기준을 설정하는 메커니즘입니다.
 - 직관적 비유: 우편물 처리 시, 너무 오래된 편지는 더 이상 유효하지 않다고 판단하여 버리는 것과 유사합니다. 기술적 설명: 스트리밍 데이터 처리에서 이벤트 시간(event time)을 기준으로 특정 지연 시간(lateness)까지 데이터를 기다려 처리하고, 그 시간을 초과한 데이터는 폐기하여 메모리 사용량을 최적화하고 정확한 집계를 보장합니다.
- 스트리밍 출력 모드 (Streaming Output Modes):
 - 업데이트 (Update): 변경된 결과만 출력합니다.
 - 덮어쓰기 (Overwrite): 매번 전체 출력을 덮어씁니다.
 - 추가 (Append): 새로운 데이터만 출력 싱크에 추가합니다.
- 처리 트리거 (Processing Trigger):
 - 한 줄 요약: 마이크로 배치(microbatch) 처리가 얼마나 자주 실행될지 결정하는 설정입니다.
 - 기술적 설명: Spark 스트리밍에서 readStream 옵션에 지정되는 간격으로, 이 간격마다 새로운 데이터 배치를 처리하도록 트리거합니다.
- 람다 vs 카파 아키텍처 (Lambda vs Kappa Architecture):
 - 람다 아키텍처 (Lambda Architecture): 배치(느린) 처리와 스트리밍(빠른) 처리를 모두 사용하여 데이터를 분석하고, 이 두 결과를 통합하여 제공하는 방식입니다. 실시간 분석과 과거 데이터 분석이 모두 필요할 때 유용합니다.
 - 카파 아키텍처 (Kappa Architecture): 모든 데이터를 단일 스트리밍 파이프라인으로 처리하는 방식입니다. 주로 실시간 처리 요구사항이 강하고, 배치 처리가 불필요하거나 스트리밍으로 대체 가능한 경우에 사용됩니다. IoT 데이터 처리나 이벤트 기반 아키텍처에 적합합니다.
- 데이터 웨어하우스 + 데이터 레이크 = 레이크하우스 (Data Warehouse + Data Lake = Lakehouse):

- **한 줄 요약:** 데이터 웨어하우스의 구조화된 관리 기능과 데이터 레이크의 유연한 대용량 저장 기능을 결합한 새로운 데이터 아키텍처입니다.
- **기술적 설명:** Delta Lake와 같은 기술을 기반으로, 데이터 레이크에 저장된 원시 데이터에 데이터 웨어하우스의 ACID 트랜잭션, 스키마 적용, 거버넌스 기능을 추가하여 데이터의 신뢰성과 활용성을 높입니다.
- **관리형 vs 외부 테이블 (Managed vs External Tables):**
 - **관리형 테이블 (Managed Tables):** 데이터와 메타데이터(스키마, 위치 등) 모두 Spark나 Hive와 같은 프레임워크에 의해 관리됩니다. 테이블을 삭제하면 데이터도 함께 삭제됩니다.
 - **외부 테이블 (External Tables):** 데이터는 외부 저장소(예: S3, HDFS)에 저장되고, 프레임워크는 메타데이터만 관리합니다. 테이블을 삭제해도 실제 데이터는 삭제되지 않고 외부 저장소에 남아 있습니다.
 - **주요 차이점:** 데이터의 물리적 저장 위치와 프레임워크에 의한 데이터 관리 범위입니다. 외부 테이블은 데이터의 독립성을 유지할 수 있어 다른 시스템과의 공유에 유리합니다.
- **메타데이터 (Metadata):**
 - **한 줄 요약:** 데이터에 대한 데이터, 즉 데이터의 내용, 구조, 출처, 소유자, 생성일 등을 설명하는 정보입니다.
 - **기술적 설명:** 메타데이터는 데이터의 발견 가능성(discoverability)과 계보(lineage)를 향상시켜 데이터 거버넌스(governance)를 강화하고, 데이터를 더 효과적으로 제어하고 관리할 수 있게 합니다.
- **구체화된 뷰 (Materialized View):**
 - **한 줄 요약:** 쿼리 결과를 미리 계산하여 물리적으로 저장해 둔 뷰입니다.
 - **기술적 설명:** 일반 뷰는 쿼리 시점에 데이터를 계산하지만, 구체화된 뷰는 미리 계산된 결과를 저장하므로 접근 속도가 매우 빠릅니다. 단점은 데이터가 새로고침되지 않으면 오래된(stale) 정보일 수 있다는 점입니다.

2.2 빅데이터 설계 패턴 (Big Data Design Patterns)

설계 패턴은 소프트웨어 공학에서 반복적으로 발생하는 문제에 대한 재사용 가능한 해결책입니다. 빅데이터 영역에서도 데이터 엔지니어링의 복잡한 문제를 해결하기 위한 특화된 설계 패턴들이 등장했습니다.

□ 핵심 요약

설계 패턴의 핵심:

- **재사용성 (Reusability):** 검증된 해결책을 반복적으로 적용하여 개발 시간과 노력을 절감합니다.
- **공통 문제 해결 (Common Problem Solving):** 데이터 엔지니어링에서 흔히 발생하는 확장성, 속도, 불륨, 처리 요구사항 등의 문제를 효과적으로 다룹니다.
- **설계 원칙 촉진 (Promotes Design Principles):** 추상화, 분할 정복, 관심사 분리, 단순성 유지 등의 원칙을 솔루션에 통합합니다.
- **공통 어휘 제공 (Provides Vocabulary):** 팀원 간 문제와 해결책을 효율적으로 논의할 수 있는 공통 언어를 제공합니다.
- **신중한 적용 (Judicious Application):** 패턴을 맹목적으로 적용하기보다는, 당면한 문제에 가장 적합한 패턴을 신중하게 선택해야 합니다.

2.2.1 설계 패턴의 종류 및 적용 영역

빅데이터 설계 패턴은 다양한 데이터 엔지니어링 과제를 해결하기 위해 등장했습니다.

- **모델링 (Modeling):**
 - **스타 및 스노우플레이크 차원 모델링 (Star and Snowflake Dimensional Modeling):** 팩트 테이블(fact table)이 차원 테이블(dimension table)로 둘러싸인 형태로, 데이터 웨어하우스에서 분석 쿼리 성능을 최적화합니다.
 - **볼트 차원 모델링 (Vault Dimensional Modeling):** 모든 원시 데이터를 볼트(vault)에 캡처한 후, 이를 기반으로 추가 데이터 처리를 수행합니다.
- **수집 (Ingestion):**
 - **커넥터 (Connectors):** 다양한 데이터 소스에 연결하여 데이터를 수집하는 표준화된 방법을 제공합니다.
 - **배치 vs 스트리밍 파이프라인 (Batch vs Streaming Pipelines):** 람다(Lambda) 및 카파(Kappa) 아키텍처와 같이, 데이터 처리의 속도와 특성에 따라 파이프라인을 설계합니다.
 - **컴퓨팅과 스토리지 분리 (Separation of Compute from Storage):** 컴퓨팅 자원과 스토리지 자원을 독립적으로 확장할 수 있도록 설계하여 효율성과 비용 효과성을 높입니다. 대량의 데이터를 저장하더라도 실제 계산이 적으면 컴퓨팅 비용을 절감할 수 있습니다.
- **변환 (Transformations):**
 - **스키마 변경 처리 (Handling Schema Changes):** 스키마 온 리드(Schema on Read)와 스키마 온 라이트(Schema on Write)와 같이, 데이터 스키마가 유동적인 경우와 고정적인 경우를 다루는 전략입니다.
 - **ACID 트랜잭션 (ACID Transactions):** 대규모 데이터에 대한 업데이트, 삭제, 병합 작업에서 데이터의 일관성과 신뢰성을 보장합니다.
 - **멀티홉 파이프라인 (Multi-hop Pipelines):** 데이터를 여러 단계로 나누어 처리하며, 각 단계에서 다양한 서비스 수준 협약(SLA)과 사용 사례를 지원합니다.
- **영속성 (Persistence):**
 - **압축된 컬럼형 형식 (Compressed Columnar Formats):** Parquet과 같이 데이터를 컬럼 단위로 압축하여 저장하여 속도와 효율성을 높이고 대량의 데이터를 처리합니다.
 - **비정규화된 와이드 테이블 (Denormalized Wide Tables):** 조인(join) 작업을 피하기 위해 데이터를 미리 비정규화하여 저장합니다.
 - **다중 목적지 (Multiple Destinations):** 데이터 파이프라인이 여러 소비자에게 다양한 요구사항에 맞춰 데이터를 분산하여 제공합니다.
- **분석 (Analytics):**
 - **인스트림 분석 (In-stream Analytics):** 데이터가 도착하는 즉시 실시간으로 분석을 수행합니다.

2.2.2 주요 빅데이터 설계 패턴 상세 설명

람다 아키텍처 (Lambda Architecture)

- **한 줄 요약:** 배치 처리와 실시간 스트리밍 처리를 결합하여, 과거 데이터와 실시간 데이터 모두에 대한 분석을 제공하는 패턴입니다.
- **구성:**
 - **배치 레이어 (Batch Layer):** 모든 과거 데이터를 처리하여 정확하고 완전한 결과를 생성합니다.

- **스피드 레이어 (Speed Layer):** 실시간으로 들어오는 데이터를 처리하여 낮은 지연 시간으로 결과를 제공합니다.
- **서빙 레이어 (Serving Layer):** 배치 레이어와 스피드 레이어의 결과를 통합하여 사용자에게 제공합니다.
- **사용 사례:** 실시간 대시보드와 함께 과거 추세 분석이 필요한 경우 (예: 웹사이트 트래픽 분석).

카파 아키텍처 (Kappa Architecture)

- **한 줄 요약:** 모든 데이터를 단일 스트리밍 파이프라인으로 처리하여, 람다 아키텍처의 복잡성을 줄인 패턴입니다.
- **구성:** 모든 데이터 처리(과거 및 실시간)가 하나의 스트리밍 파이프라인에서 이루어집니다.
- **사용 사례:** IoT 데이터 처리, 이벤트 기반 시스템, 실시간 응답이 중요한 애플리케이션.

이벤트 기반 아키텍처 (Event-Driven Architecture)

- **한 줄 요약:** 시스템 구성 요소들이 이벤트의 발생과 소비를 통해 비동기적으로 상호작용하는 패턴입니다.
- **구성:** 이벤트 생산자(Event Producers), 이벤트 스트리밍(Event Streaming), 이벤트 소비자(Event Consumers)로 이루어집니다.
- **사용 사례:** 전자상거래 거래 처리, 실시간 알림 시스템.

CQRS (Command Query Responsibility Segregation)

- **한 줄 요약:** 데이터 쓰기(Command)와 읽기(Query)의 책임을 분리하여 시스템을 설계하는 패턴입니다.
- **설명:**
 - **쓰기 최적화 (Write Optimization):** 데이터 입력 및 업데이트를 효율적으로 처리하도록 시스템을 설계합니다.
 - **읽기 최적화 (Query Optimization):** 데이터 조회(쿼리)를 빠르고 효율적으로 처리하도록 시스템을 설계합니다.
- **장점:** 쓰기 및 읽기 부하에 따라 시스템을 독립적으로 확장할 수 있어, 높은 데이터 처리량과 빠른 쿼리 응답 시간을 동시에 달성할 수 있습니다.

폴리글랏 영속성 (Polyglot Persistence)

- **한 줄 요약:** 데이터의 특성과 사용 목적에 따라 가장 적합한 여러 종류의 데이터베이스를 혼합하여 사용하는 패턴입니다.
- **직관적 비유:** "망치만 있으면 모든 것이 못으로 보인다"는 말처럼, 하나의 데이터베이스만 고집하는 것이 아니라, 작업에 맞는 올바른 도구를 선택하는 것과 같습니다.
- **사용 사례:**
 - **관계형 데이터베이스 (Relational Databases):** 트랜잭션 데이터.
 - **문서 저장소 (Document Stores):** 비정형 또는 반정형 데이터.
 - **그래프 데이터베이스 (Graph Databases):** 관계 중심 데이터.

- **장점:** 각 데이터베이스의 강점을 활용하여 시스템의 전반적인 성능과 유연성을 극대화합니다.

데이터 레이크 패턴 (Data Lake Pattern)

- **한 줄 요약:** 대량의 원시 데이터를 어떤 변환도 거치지 않은 원래 형태로 저장하는 패턴입니다.
- **설명:** 데이터를 수집할 때 정확히 어떻게 사용될지 모르는 경우, 모든 데이터를 원본 형태로 저장하여 나중에 다양한 분석 목적으로 재탐색할 수 있도록 합니다.
- **저장:** 정형 및 비정형 데이터를 모두 저장합니다.
- **처리 도구:** Hadoop, Spark 등을 사용하여 저장된 데이터를 처리합니다.
- **장점:** 미래의 알 수 없는 사용 사례에 대비하여 데이터의 잠재적 가치를 보존합니다.

마이크로 배치 처리 (Microbatch Processing)

- **한 줄 요약:** 스트리밍 데이터를 작은 배치 단위로 나누어 주기적으로 처리하는 패턴입니다.
- **설명:** 연속적인 데이터 스트림을 실시간에 가까운(near real-time) 통찰력을 얻기 위해 작은 시간 간격(예: 1초)으로 데이터를 수집하고 처리합니다.
- **특징:**
 - **정확히 한 번 처리 (Exactly-once Semantics):** 데이터가 중복 처리되지 않도록 보장합니다.
 - **장애 복원력 (Resilient to Failures):** 처리 중 오류가 발생해도 복구할 수 있습니다.
 - **인스트림 분석 (In-stream Analytics):** 데이터가 도착하는 즉시 분석을 수행합니다.
 - **지연 데이터 처리 (Handling Late Arriving Data):** 워터마킹 등을 통해 지연 도착 데이터를 처리합니다.

CAP 이론 (CAP Theorem)

- **한 줄 요약:** 분산 시스템은 일관성(Consistency), 가용성(Availability), 분할 내성(Partition Tolerance) 중 최대 두 가지만 동시에 만족할 수 있다는 이론입니다.
- **세 가지 요소:**
 - **일관성 (Consistency):** 모든 노드에서 동일한 시점에 동일한 데이터를 볼 수 있습니다.
 - **가용성 (Availability):** 모든 요청에 대해 항상 응답을 받을 수 있습니다 (오류가 발생하더라도).
 - **분할 내성 (Partition Tolerance):** 네트워크 분할(노드 간 통신 단절)이 발생해도 시스템이 계속 작동합니다.
- **선택:**
 - **CP 기반 시스템 (Consistency over Availability):** 일관성을 우선시합니다 (예: 관계형 데이터베이스). 네트워크 분할 시 시스템이 일시적으로 작동을 멈출 수 있습니다.
 - **AP 기반 시스템 (Availability over Consistency):** 가용성을 우선시합니다 (예: NoSQL 데이터베이스). 네트워크 분할 시에도 계속 작동하지만, 데이터 일관성이 일시적으로 깨질 수 있습니다. 빅데이터 시스템은 대규모 데이터 처리 요구사항 때문에 AP 시스템으로 진화하는 경우가 많습니다.

제로 데이터 손실 / 제로 클론 패턴 (Zero Data Loss / Zero Clone Pattern)

- **한 줄 요약:** 데이터 손실을 최소화하고 데이터 복제를 효율적으로 관리하는 패턴입니다.
- **설명:** 분산 시스템(예: Kafka)에서 데이터를 내부적으로 복제하여, 클러스터의 한 노드가 손실되어도

데이터가 다른 노드에 남아있도록 합니다. 이는 금융 시스템이나 의료 시스템처럼 데이터 손실이 치명적인 경우에 중요합니다.

데이터 메시 (Data Mesh)

- **한 줄 요약:** 대규모 조직에서 데이터를 제품처럼 취급하고, 각 도메인 팀이 데이터 파이프라인을 독립적으로 관리하며 표준화된 방식으로 데이터를 공유하는 아키텍처입니다.
- **특징:**
 - 각 팀은 자체 데이터 파이프라인을 엔드 투 엔드로 관리합니다.
 - 조직 전체의 표준화된 도구와 방법을 사용합니다.
 - 생성된 데이터는 데이터 제품(Data Product)으로 공유되며, 내부 및 외부 사용자에게 소비됩니다.
- **장점:** 데이터 소유권과 책임감을 명확히 하고, 데이터 활용을 가속화합니다.

에이전트 아키텍처 (Agentic Architecture)

- **한 줄 요약:** 대규모 언어 모델(LLM)을 활용하여 다양한 도구와 에이전트의 작업을 조율하여 복잡한 태스크를 수행하는 최신 설계 패턴입니다.
- **설명:** 에이전트가 외부 API나 데이터베이스를 쿼리하여 특정 작업을 수행하고, 여러 에이전트가 협력하여 더 큰 솔루션을 제공합니다. AI와 기존 시스템을 결합하는 흥미로운 방법입니다.

커넥터 패턴 (Connector Pattern)

- **한 줄 요약:** 다양한 데이터 소스에 일관된 방식으로 접근할 수 있도록 추상화된 인터페이스를 제공하는 패턴입니다.
- **소프트웨어 공학 용어:** 브릿지 패턴(Bridge Pattern)이라고도 합니다.
- **예시:** JDBC(Java Database Connectivity) 커넥터는 다양한 관계형 데이터베이스에 동일한 Java API를 통해 접근할 수 있게 합니다. Spark SQL이 JDBC/ODBC 커넥터를 통해 다양한 데이터 저장소와 상호작용하는 것이 좋은 예시입니다.
- **장점:** 소비자는 데이터 소스의 종류에 관계없이 일관된 방식으로 데이터를 처리할 수 있습니다.

멀티홉 / 메달리온 아키텍처 (Multi-hop / Medallion Architecture)

- **한 줄 요약:** 데이터 파이프라인을 여러 논리적 단계(홉)로 나누어 데이터 품질을 점진적으로 향상시키고, 각 단계에서 다양한 사용자 그룹의 요구사항을 충족시키는 패턴입니다.
- **단계:**
 - **랜딩 영역 (Landing Area) / 브론즈 (Bronze):** 원시 데이터를 저장합니다. 데이터 엔지니어가 주로 사용합니다.
 - **정제 영역 (Refined Area) / 실버 (Silver):** 원시 데이터를 정제하고 구조화합니다. 데이터 과학자가 주로 사용합니다.
 - **집계 영역 (Aggregation Area) / 골드 (Gold):** 비즈니스 분석가가 바로 사용할 수 있도록 데이터를 집계하고 최적화합니다.
- **특징:**
 - **데이터 품질 증가:** 각 홉을 거칠수록 데이터 품질이 향상됩니다.

- **데이터 가용성 증가:** 초기 흡일수록 데이터 가용성이 높습니다 (데이터가 빨리 도착).
- **설계 원칙 적용:** 분할 정복(divide and conquer) 및 관심사 분리(separation of concerns) 원칙을 적용하여 파이프라인 설계를 단순화합니다.

데이터 이동 최소화 (Minimizing Movement of Data) 데이터 이동은 비용과 시간을 소모하므로, 이를 최소화하는 기술들이 중요합니다.

- **시간 여행 (Time Travel):**

- **한 줄 요약:** 데이터의 특정 과거 시점 또는 버전으로 돌아가 데이터를 조회하거나 복원하는 기능입니다.
- **기술적 설명:** Delta Lake는 Parquet 형식 위에 변경 이력을 기록하는 저널 파일(journal file)을 관리하여, 데이터의 특정 버전으로 쉽게 되돌아갈 수 있게 합니다. 이는 데이터 재처리나 오류 복구에 매우 유용합니다.

- **제로 카피 클론 (Zero Copy Clone):**

- **한 줄 요약:** 실제 데이터를 복사하지 않고, 메타데이터만 복사하여 데이터의 복제본을 생성하는 기술입니다.
- **기술적 설명:** 새로운 메타데이터 세트만 생성하고 원본 데이터는 그대로 유지하므로, 저장 공간을 절약하고 복제 속도를 크게 향상시킵니다.
- **Delta Clone:**
 - * **얕은 클론 (Shallow Clone):** 제로 카피 클론과 동일하게 메타데이터만 복사합니다.
 - * **깊은 클론 (Deep Clone):** 메타데이터와 실제 데이터를 모두 복사합니다.

- **Delta Share:**

- **한 줄 요약:** 데이터 복사 없이 외부 사용자에게 데이터 접근 권한을 공유하는 방법입니다.
- **장점:** 저장 비용, 데이터 접근 비용, 네트워크 전송 비용을 절감하고, 원본 데이터의 일관성을 유지합니다.

2.3 데이터 규정 준수 (Data Compliance)

데이터 엔지니어로서 데이터 규정 준수는 매우 중요한 책임입니다. 개인 정보 보호와 관련된 법규를 이해하고 준수해야 합니다.

□ 핵심 요약

데이터 규정 준수의 중요성:

- **소비자 권리 보호:** 개인 데이터에 대한 소비자의 권리를 보호합니다.
- **법적 책임:** 규정 위반 시 상당한 벌금이 부과될 수 있습니다.
- **데이터 관리 책임:** 데이터 관리자로서 데이터를 책임감 있게 관리해야 합니다.
- **설계 고려사항:** 데이터 파이프라인 및 시스템 설계 시 규정 준수 요건을 미리 반영해야 합니다.

2.3.1 주요 규정

- **GDPR (General Data Protection Regulation):**

- **한 줄 요약:** 유럽 연합(EU)의 일반 데이터 보호 규정으로, EU 시민의 개인 데이터 보호를 강화합니다.

- **적용 범위:** 데이터가 미국에 저장되어 있더라도, EU 시민의 데이터라면 GDPR이 적용됩니다.
- **CCPA (California Consumer Privacy Act):**
 - **한 줄 요약:** 캘리포니아 소비자 개인 정보 보호법으로, 캘리포니아 주민의 개인 정보 보호를 위한 프레임워크를 제공합니다.

2.3.2 핵심 소비자 권리

- **삭제할 권리 (Right of Erasure) / 잊힐 권리 (Right to be Forgotten):**
 - **한 줄 요약:** 소비자가 자신의 개인 데이터를 시스템에서 삭제하도록 요구할 수 있는 권리입니다.
 - **기술적 과제:** 대규모 데이터 레이크에서 특정 사용자의 데이터를 식별하고, 다른 사용자 데이터에 영향을 주지 않으면서 해당 데이터를 완전히 삭제하는 것은 매우 어려운 작업입니다.
- **데이터 이동성 권리 (Right of Portability):**
 - **한 줄 요약:** 소비자가 자신의 데이터를 시스템에서 추출하여 다른 서비스(경쟁사 포함)로 이전하도록 요구할 수 있는 권리입니다.
 - **기술적 과제:** 특정 사용자의 데이터를 식별하고, 시스템에서 추출하여 표준화된 형식으로 제공할 수 있어야 합니다.

2.3.3 규정 준수를 위한 기술적 접근

- **세분화된 업데이트 및 삭제 (Fine-grained Updates and Deletes):** Delta Lake와 같은 기술은 대규모 데이터에서 특정 행에 대한 업데이트 및 삭제를 지원하여 규정 준수를 용이하게 합니다.
- **가명화 / 익명화 (Pseudonymization / Anonymization):**
 - **한 줄 요약:** 개인 식별이 가능한 정보를 비식별화된 식별자(예: 토큰, GUID)로 대체하여 프라이버시를 보호하는 방법입니다.
 - **기술적 설명:** 실제 사용자 정보와 비식별화된 식별자 간의 매핑 정보를 별도로 관리합니다. 사용자의 데이터 삭제 요청 시, 이 매핑 정보만 삭제하면 실제 데이터는 시스템에 남아있더라도 더 이상 특정 사용자를 추적할 수 없게 됩니다.

주의사항

규정 준수 유의사항: 의료 분야(HIPAA)나 정부 기관(FedRAMP)과 같은 규제 산업에서는 데이터 보호 및 보안 표준이 훨씬 엄격합니다. 데이터 엔지니어는 이러한 추가적인 요구사항을 반드시 인지하고 시스템 설계에 반영해야 합니다. 데이터 분류, 접근 감사 추적 등 추가적인 계층이 필요합니다.

3 Spark 내부 동작 및 최적화

Spark는 분산 컴퓨팅 프레임워크로, 대규모 데이터 처리를 위해 복잡한 내부 메커니즘을 가지고 있습니다. Spark의 동작 방식을 이해하는 것은 성능 최적화에 필수적입니다.

3.1 Spark 아키텍처: 작업, 단계, 태스크 (Jobs, Stages, Tasks)

Spark는 사용자의 코드를 여러 단계로 나누어 분산 환경에서 병렬로 실행합니다.

- **드라이버 (Driver):** Spark 애플리케이션의 메인 프로세스로, 작업을 스케줄링하고 워커 노드(Worker Nodes)에 태스크를 분배합니다.
- **워커 노드 (Worker Nodes):** 실제 데이터 처리 작업을 수행하는 물리적 또는 가상 머신입니다. 각 워커 노드에는 하나 이상의 실행자(Executor)가 있습니다.
- **실행자 (Executor):** 워커 노드에서 Spark 태스크를 실행하는 JVM 프로세스입니다.

Figure 1: Spark 아키텍처: 드라이버, 워커, 실행자, 작업, 단계, 태스크

- **작업 (Job):** Spark 액션(예: `collect()`, `write()`)이 트리거될 때 생성됩니다. 하나의 Spark 애플리케이션은 여러 작업을 포함할 수 있습니다.
- **단계 (Stage):** 작업은 여러 단계로 나뉩니다. 각 단계는 논리적인 작업 단위로, 데이터의 셔플(shuffle) 발생 여부에 따라 구분됩니다.
 - **좁은 변환 (Narrow Transformation):** `map`, `filter`와 같이 각 파티션의 데이터만으로 결과를 계산할 수 있는 변환입니다. 셔플이 발생하지 않습니다.
 - **넓은 변환 (Wide Transformation) / 셔플 변환 (Shuffle Transformation):** `groupBy`, `reduceBy`와 같이 다른 파티션의 데이터가 필요한 변환입니다. 데이터 셔플이 발생하며, 이는 네트워크 오버헤드를 유발하여 비용이 많이 드는 작업입니다. 셔플은 최소화하는 것이 좋습니다.
- **태스크 (Task):** 단계는 다시 여러 태스크로 나뉩니다. 태스크는 Spark에서 실행되는 가장 작은 작업 단위이며, 실행자의 코어(core)에서 병렬로 실행됩니다.

□ 예제

Spark 쿼리 실행 흐름:

1. 사용자가 Python(PySpark) 또는 SQL 코드를 작성합니다.
2. 코드는 Spark 드라이버로 제출됩니다.
3. 드라이버는 코드를 여러 **작업(Jobs)**으로 나눕니다.
4. 각 작업은 Spark 엔진에 의해 최적화 과정을 거쳐 여러 **단계(Stages)**로 분할됩니다. 셔플이 필요한 지점에서 새로운 단계가 시작됩니다.
5. 각 단계는 여러 **태스크(Tasks)**로 분할됩니다.
6. 태스크는 워커 노드의 **실행자(Executors)**에 할당된 코어에서 병렬로 실행됩니다.
7. 태스크는 데이터의 작은 부분을 처리하고, 결과는 다시 드라이버를 거쳐 사용자에게 전달됩니다.

3.2 테이블 파티셔닝 vs Spark 파티셔닝

‘파티션’이라는 용어는 Spark 생태계에서 여러 맥락에서 사용되므로 혼동하기 쉽습니다. 테이블 파티셔닝과 Spark 파티셔닝은 서로 다른 개념입니다.

Table 1: 테이블 파티셔닝과 Spark 파티셔닝 비교

구분	테이블 파티셔닝 (Table Partitioning)	Spark 파티셔닝 (Spark Partitioning)
정의	대규모 테이블을 디스크 상의 물리적인 작은 조각(파일/디렉토리)으로 나누는 방식	Spark가 데이터를 워커 노드에 분산하여 처리하는 논리적인 방식
목적	데이터 조직화, 쿼리 성능 향상 (디스크 I/O 감소)	병렬 처리, 최적화, 장애 허용
제어 주체	사용자 (DDL 문에서 명시적으로 지정)	Spark 엔진 (데이터, 쿼리, 설정에 따라 동적으로 결정)
예시	PARTITIONED BY (country, date)	Spark 데이터프레임의 내부 파티션 수
최적화 기법	파티션 가지치기 (Partition Pruning): WHERE 절에 파티션 키를 사용하면 해당 파티션만 스캔하여 불필요한 데이터 읽기 방지	데이터 셔플링 (Data Shuffling): 조인, 그룹화 시 데이터 재분배. 최소화하는 것이 성능에 중요
변경 용이성	일단 설정되면 변경하기 어려움 (데이터 재구성 필요)	쿼리 실행 시 동적으로 변경 가능

3.2.1 테이블 파티셔닝 (Table Partitioning)

- **한 줄 요약:** 대규모 테이블을 디스크 상의 물리적인 작은 조각으로 나누어 관리하고 쿼리 성능을 높이는 기법입니다.
- **필요성:** 데이터가 너무 많아 전체 테이블을 스캔하는 데 시간이 오래 걸릴 때, 특정 조건의 데이터만 빠르게 찾기 위해 사용합니다.
- **예시:** 판매 데이터를 country나 date 컬럼으로 파티셔닝하면, 특정 국가나 날짜의 데이터만 조회할 때 전체 데이터를 스캔할 필요 없이 해당 파티션만 읽어 빠르게 결과를 얻을 수 있습니다.
- **파티션 가지치기 (Partition Pruning):** WHERE country = 'USA' 와 같은 쿼리가 들어오면, Spark는 'USA' 파티션만 읽어 불필요한 디스크 I/O를 줄입니다.
- **파티셔닝 키 선택 기준:**
 - WHERE 절에서 자주 사용되는 컬럼을 선택합니다.
 - 카디널리티(Cardinality, 고유 값의 수)가 너무 높지 않은 컬럼을 선택합니다. (너무 많으면 작은 파일이 너무 많이 생성됨)
 - 각 파티션에 최소 1GB 이상의 데이터가 있도록 하는 것이 좋습니다.
 - **최신 권장 사항:** 데이터 크기가 1TB 미만인 경우, 수동 파티셔닝을 하지 않는 것이 좋습니다. Spark 엔진의 내장된 최적화 기능이 더 효율적일 수 있습니다. 잘못된 파티셔닝은 오히려 성능을 저하시킬 수 있습니다.

□ 예제

SQL을 이용한 테이블 파티셔닝 예시:

```

1 CREATE TABLE customer (
2   id INT,
3   name STRING,
4   age INT,
5   city STRING,
6   state STRING
7 )
8 PARTITIONED BY (state, city); -- 와 state city 컬럼으로 파티셔닝
9
10 INSERT INTO customer PARTITION (state='CA', city='Los Angeles')
11 VALUES (1, 'Alice', 30); -- 특정 파티션에 데이터 삽입
12
13 SHOW PARTITIONS customer; -- 파티션 목록 조회
14 DESCRIBE FORMATTED customer; -- 테이블 상세 정보 파티션 (포함)

```

Listing 1: 테이블 파티셔닝 정의

3.2.2 Spark 파티셔닝 (Spark Partitioning)

- **한 줄 요약:** Spark가 데이터를 워커 노드에 분산하여 병렬 처리하기 위한 논리적인 데이터 분할 단위입니다.
- **필요성:** Spark의 분산 컴퓨팅 능력을 최대한 활용하여 대규모 데이터를 빠르게 처리하고, 장애 발생 시 복원력을 높이기 위함입니다.
- **제어:** Spark는 입력 데이터의 크기, Spark 설정, 데이터 스큐(skew) 등을 기반으로 파티션 수를 동적으로 결정합니다. 사용자가 직접 파티션 수를 조절할 수도 있지만, 신중하게 접근해야 합니다.
- **데이터 셔플링 (Data Shuffling):** Spark 파티션 간에 데이터가 재분배되는 과정으로, 조인(JOIN), 그룹화(GROUP BY), 축소(REDUCE BY)와 같은 넓은 변환에서 발생합니다. 셔플은 네트워크 오버헤드를 유발하므로 최소화해야 합니다.

주의사항

파티셔닝 관련 흔한 실수:

- **너무 적은 파티션:** 병렬 처리의 이점을 충분히 활용하지 못하고, 일부 워커 노드에 과부하가 걸릴 수 있습니다.
- **너무 많은 파티션:** 작은 파일이 너무 많이 생성되어 파일 접근 오버헤드가 커지고, Spark의 스케줄링 및 관리 비용이 증가하여 오히려 성능이 저하될 수 있습니다. 각 파티션에 최소 1GB의 데이터가 있도록 하는 것이 좋습니다.

3.3 쿼리 최적화 과정

Spark를 포함한 대부분의 데이터베이스 시스템은 쿼리를 실행하기 전에 복잡한 최적화 과정을 거칩니다.

1. **논리적 계획 (Logical Plan):** 사용자가 제출한 쿼리(SQL 또는 DataFrame API)는 먼저 추상적인